

Final exam: HTML, CSS and JavaScript

Name _____

Date _____

Section	What it is	Marks	Conditions
A	30 objective, 3 theory questions	30	45 minutes, on paper, no notes or devices
B	Practical project	70	Notes and MDN allowed. Deadline: _____

Section A: Questions

30 marks

PART 1: OBJECTIVE QUESTIONS

15 marks

Each question is worth half a mark. Circle one letter.

HTML

- Your page `index.html` sits in the main project folder. The logo is saved at `assets/images/logo.png`. Which tag shows the logo and gives a screen reader something to say about it?

 - ``
 - ``
 - ``
 - `<img href="assets/images/logo.png" alt="The shop's logo">`
- An "Add to cart" control changes a number on the same page. It does not take you anywhere. Which element should it be?

 - `<button>`
 - `<a href="#">`
 - a `<div>` with a click event
 - `<span class="btn">`
- Which of these is nested correctly?

 - `<p>Price: <strong>#2,500</p></strong>`
 - `<ul><p>Jollof rice</p></ul>`
 - `<li><ul>Jollof rice</ul></li>`
 - `<ul><li>Jollof rice <strong>(new)</strong></li></ul>`
- The browser tab for your page says "Document". What do you change to fix it?

 - the `<h1>`
 - the `<title>` inside `<head>`
 - the name of the file
 - the `lang` attribute on `<html>`

5. You are listing the steps of a recipe, and they must be followed in order. Which element should hold the steps?
- (a) `<ul>`, with an `<li>` for each step
 - (b) one `<p>`, with the steps separated by commas
 - (c) an `<li>` for each step, with nothing around them
 - (d) `<ol>`, with an `<li>` for each step
6. The headings on a page come in this order. What is wrong?
- ```
<h1>Our menu</h1>
<h3>Rice dishes</h3>
<h2>Soups</h2>
```
- (a) a level is skipped: "Rice dishes" should be an `<h2>`, like "Soups"
  - (b) nothing, because heading levels can be used in any order
  - (c) a page needs more than one `<h1>`
  - (d) every heading level from `h1` to `h6` must be used once
7. You want a link in the nav that jumps down to the Prices section on the same page. Which pair works?
- (a) `<a href="#prices">` and `<section class="prices">`
  - (b) `<a href="prices">` and `<section id="prices">`
  - (c) `<a href="#prices">` and `<section id="prices">`
  - (d) `<a href=".prices">` and `<section class="prices">`
8. Two pages look exactly the same in the browser. One is built with `<header>`, `<nav>`, `<main>` and `<footer>`. The other uses only `<div>`. What does the first page gain?
- (a) it loads faster
  - (b) it gets colours and spacing without any CSS
  - (c) nothing, it is only a matter of taste
  - (d) screen readers and search engines can tell what each part of the page is

## CSS

9. `index.html` is in the main project folder and the stylesheet is at `css/styles.css`. Which line links it correctly?
- (a) `<link rel="stylesheet" href="css/styles.css">`
  - (b) `<link rel="stylesheet" href="/styles.css">`
  - (c) `<style src="css/styles.css"></style>`
  - (d) `<script src="css/styles.css"></script>`

10. What colour is the text?

```
p { color: black; }
.note { color: green; }
#intro { color: red; }
.note { color: blue; }

<p id="intro" class="note">Doors open at 4pm.</p>
```

- (a) black                      (b) green                      (c) red                      (d) blue

11. What colour is the text?

```
p.price { color: green; }
.price { color: blue; }

<p class="price">£2,500</p>
```

- (a) green, because p.price is more specific                      (b) blue, because it comes last  
(c) black, because the two rules cancel out                      (d) it depends on the browser

12. With the default box-sizing, how wide is this card from the outside of its left border to the outside of its right border?

```
.card {
 width: 300px;
 padding: 20px;
 border: 5px solid black;
 margin: 10px;
}
```

- (a) 300px                      (b) 340px                      (c) 350px                      (d) 370px

13. A card has a background colour, and its text sits right against the edge of that colour. Which property adds space between the text and the edge?

- (a) margin                      (b) padding                      (c) border                      (d) gap

14. What does the selector `nav a:hover` pick out?

- (a) every nav, and every link, while the pointer is over them  
(b) any link inside a nav, while the pointer is over that link  
(c) the nav itself, when any link inside it is hovered  
(d) only the first link in the nav

15. An image with `transform: rotate(10deg)` turns around its centre. You want it to swing from the middle of its bottom edge instead, like a card fanned out in your hand. Which line do you add?

- (a) `rotate-from: bottom;`                      (b) `transform: rotate(10deg) bottom;`  
(c) `position: bottom center;`                      (d) `transform-origin: bottom center;`

- 16.** A big decorative circle inside a section sticks out past the section's edges. Which rule, on the section, hides the parts that stick out?
- (a) `overflow: hidden;`                  (b) `display: none;`                  (c) `z-index: -1;`                  (d) `visibility: hidden;`
- 17.** You want a coloured shape behind your heading to look soft and out of focus, like a glow. Which line does that?
- (a) `opacity: 0.5;`                                                  (b) `filter: blur(40px);`  
(c) `border-radius: 50%;`                                                  (d) `transform: scale(2);`
- 18.** In the DevTools Styles panel, the color line in one of your rules has a line drawn through it. What is DevTools telling you?
- (a) the browser does not support colour                          (b) the stylesheet failed to load  
(c) another rule is overriding it                                      (d) the element is hidden

# JavaScript

- 19.** What does this print?

```
console.log("5" + 3, "5" - 3);
```

- (a) 8 2                      (b) 53 53                      (c) 8 8                      (d) 53 2

- 20.** What happens when this runs?

```
const score = 10;
score = score + 5;
```

- (a) an error: assignment to constant variable      (b) score becomes 15  
(c) score stays 10, with no error      (d) score becomes "105"

- 21.** With `const name = "Tobi";` and `const items = 3;`, which line prints Tobi ordered 3 items?

- (a) `console.log(`${name} ordered ${items} items`);`
- (b) `console.log(`$name ordered $items items`);`
- (c) `console.log(`${name} ordered ${items} items`);`
- (d) `console.log(`{name} ordered {items} items`);`

- 22.** What appears in the console, in order?

```
function double(n) {
 console.log(n * 2);
}

const result = double(4);
console.log(result);
```

- (a) 8, then 8                      (b) 8, then undefined                      (c) undefined, then 8                      (d) only 8

**23.** Which of these is true?

- (a) `"7" === 7`                      (b) `0 === ""`                      (c) `7 === "7"`                      (d) `"7" == 7`

**24.** What does `canEnter(20, false)` return?

```
function canEnter(age, hasTicket) {
 if (age >= 18 && hasTicket) {
 return "Welcome";
 }
 if (age >= 18 || hasTicket) {
 return "Almost";
 }
 return "Sorry";
}
```

- (a) "Welcome"                      (b) "Almost"                      (c) "Sorry"                      (d) undefined

**25.** What does this print?

```
const prices = [1200, 800, 2500, 500];
let total = 0;

for (let i = 0; i < prices.length; i++) {
 if (prices[i] >= 1000) {
 total = total + prices[i];
 }
}

console.log(total);
```

- (a) 3700                      (b) 5000                      (c) 2500                      (d) 1200

**26.** What does this print?

```
const team = ["Ada", "Bayo"];
team.push("Chi");
console.log(team.length, team[1]);
```

- (a) 2 Bayo                      (b) 3 Chi                      (c) 3 Bayo                      (d) 3 Ada

**27.** This prints three numbers, then stops with an error. Why?

```
const names = ["Ada", "Bayo", "Chi"];

for (let i = 0; i <= names.length; i++) {
 console.log(names[i].length);
}
```

- (a) `names` is a `const`, so the loop cannot read it  
(b) `.length` does not work on strings  
(c) `<=` makes the loop run once too often, and `names[3]` is undefined  
(d) `i` should start at 1

- 28.** The console shows Cannot read properties of null (reading 'addEventListener'). What is the cause?

```
<button class="buy-btn">Buy now</button>

const button = document.querySelector("#buy-btn");
button.addEventListener("click", function () {
 // ...
});
```

- (a) addEventListener is spelled wrong
  - (b) the function needs a name
  - (c) buttons cannot listen for clicks
  - (d) #buy-btn looks for an id, but the button only has a class
- 29.** This runs with no errors, but the message never turns red. Why?

```
/* styles.css */
.error { color: red; }

// script.js
message.classList.add(".error");
```

- (a) classList.add only works on buttons
  - (b) it adds a class literally called ".error", which the .error rule does not match
  - (c) the CSS rule needs !important
  - (d) message must be declared with let
- 30.** The button is clicked three times. What does label show after the third click?

```
button.addEventListener("click", function () {
 let count = 0;
 count = count + 1;
 label.textContent = count;
});
```

- (a) 1                      (b) 0                      (c) 3                      (d) 123

## PART 2: THEORY

15 marks

Each question is worth 5 marks. Write your answer in the space given.

- 31.** Write the CSS that puts these three cards side by side in a row with 20px of space between them. Each card should also have 16px of space between its text and its edge, and a 1px solid grey border.

```
<div class="cards">
 <div class="card">...</div>
 <div class="card">...</div>
 <div class="card">...</div>
</div>
```

---

---

---

---

---

---

---

---

- 32.** Write a loop that logs the name of every student who scored 50 or more. It should print Ada, then Zainab.

```
const students = [
 { name: "Ada", score: 72 },
 { name: "Emeka", score: 45 },
 { name: "Zainab", score: 88 }
];
```

---

---

---

---

---

---

---

---

- 33.** Whatever the user types, clicking Go always shows "Type your name first". Explain the bug and show the fix.

```
<input id="name">
<button id="go">Go</button>
<p id="msg"></p>

const input = document.querySelector("#name");
const msg = document.querySelector("#msg");
const typed = input.value;

document.querySelector("#go").addEventListener("click", function () {
 if (typed === "") {
 msg.textContent = "Type your name first";
 } else {
 msg.textContent = `Welcome, ${typed}`;
 }
});
```

---

---

---

---

---

---

---

---

### A ticket page for a one-day event

Make up a one-day event: a tech meetup, a gaming tournament, a small concert, a football viewing party, a class reunion, anything you like. Then build a one-page website for it. The page tells people what the event is and when things happen, and it has a booking panel where they pick tickets, see the price add up, and book.

No real payments happen and nothing is saved. Everything runs on the page, in the browser, with your own HTML, CSS and JavaScript. You can build all of it in a phone editor such as Acode or Spck, or on a laptop with VS Code.

### Your project folder

```
event/
 index.html
 styles.css
 script.js
 notes.txt
 images/
```

### Part 1: HTML

14 marks

The page needs a correct skeleton: doctype, lang, charset and a real title. Inside the body you need:

- a `<header>` with the event's name and a `<nav>` holding at least three links. Each link jumps down to a different section of the page, for example one to About, one to Schedule and one to Tickets.
- a `<main>` with at least four sections: About, Schedule, Tickets, and one more of your choice (FAQ, Speakers, Line-up or Venue).
- a schedule written as an ordered list with at least four items, each with a time.
- a `<footer>` with your name and the event date.
- exactly one `<h1>`, and no skipped heading levels.

The Tickets section holds the booking panel. It has three ticket types. Each one shows its name, its price, a minus button, the number chosen and a plus button. Under them goes the total, a name input, a phone number input with a label for each, a Book button, a Clear button and an empty paragraph for messages. Anything that does something on the same page is a `<button>`. Anything that goes somewhere is a link.

What is checked	Marks
Skeleton and head are complete and correct	2
Header, nav, main, sections and footer used for the right jobs	3
Every nav link lands on its section	2
Schedule as an ordered list, with a time on every item	2
Booking panel complete, with inputs labelled and buttons used for actions	3
One h1, sensible heading order, every tag closed and nested correctly	2

## Part 2: CSS

18 marks

All styling goes in `styles.css`. Do not use `style=""` attributes. Pick a small set of colours and one or two fonts and stick to them. Text must be easy to read against its background.

- Give the ticket types and other content blocks proper spacing with padding and margin, and set `box-sizing: border-box` on everything.
- Use Flexbox in three places: the header, with the event name on one side and the nav on the other; the nav links, in a row; and the three ticket types, side by side.
- Style each ticket type as a card with a border or background, and make the buttons look like buttons someone would want to press.
- Buttons and links change in some visible way on hover, and show a clear outline when you reach them with the keyboard (`:focus`).
- Use at least one technique from the CSS techniques session somewhere it makes sense: `transform` with `transform-origin`, a mask-image fade, a `filter: blur()` glow, or `overflow: hidden` to crop something on purpose.

What is checked	Marks
One linked stylesheet, no inline styles	1
Consistent colours and fonts, readable contrast	2
Spacing with padding and margin, border-box set	3
Flexbox for the header, the nav and the ticket types	5
Ticket cards and buttons styled clearly	3
Hover and focus styles	1
A technique from the CSS techniques session, used with a reason	3

## Part 3: JavaScript

28 marks

Start `script.js` with your tickets stored as an array of three objects. Change the names and prices if they do not fit your event.

```
const tickets = [
 { name: "Regular", price: 3000, count: 0 },
 { name: "VIP", price: 10000, count: 0 },
 { name: "Student", price: 1500, count: 0 }
];
```

The total must always be worked out from this array. If you change a price here, the total on the page must change with it.

The plus button adds one ticket of that type, and the minus button takes one away. A count can never go below 0. The whole order can have at most 10 tickets. If someone presses plus when they already have 10, the count stays the same and the message paragraph says "You can book at most 10 tickets." The numbers on the page update every time.

Write a function that loops through the array and *returns* the total price of the order. Use it to update the total on the page after every change. The total can show as plain digits, so "16500" is fine.

When the order has 5 or more tickets, take 10% off. Show the price before the discount, the price after it, and the words "Group discount applied". For example, 2 Regular and 3 Student is 5 tickets: ₦10,500 before the discount and ₦9,450 after it.

When the Book button is clicked, read the name and phone number at that moment and check them in this order:

1. If no tickets are chosen, show "Pick at least one ticket."
2. If the name is shorter than 2 characters, show "Enter your name."
3. If the phone number is not exactly 11 characters long, show "Enter an 11-digit phone number."

Show only the first problem found, and give the message paragraph the class error so it looks like a warning. If everything passes, remove that class, add the class success, and use a template literal to show a message like this:

```
Thanks, Ada. 5 tickets booked for ₦9450. We will text 08031234567.
```

The Clear button sets every count back to 0, empties both inputs and the message, and updates the page.

Finally, make one more part of the page respond to a click. For example, clicking an FAQ question shows or hides its answer, or clicking a speaker's name shows their short bio. Do it by adding and removing a class, and let your CSS do the hiding and showing.

What is checked	Marks
Tickets stored as an array of objects, and the total comes from it	3
Plus and minus work, never below 0, never above 10 in total, with the message	6
A function that loops and returns the total, shown after every change	6
Group discount from 5 tickets, both prices and the words shown	3
Book checks in the right order, reads the inputs on click, uses the error and success classes	6
Clear resets the counts, inputs, message and total	2
One more click interaction, done with a class	2

#### Part 4: Quality and your notes

10 marks

Keep your code tidy: indent it properly and give things names that say what they hold, such as `totalPrice` rather than `x`. Open the console and use every part of your page. No red errors should appear.

In `notes.txt`, answer these three questions in your own words. A few honest sentences each is enough.

1. Describe one bug you hit while building this. What did you see, how did you find the cause, and what fixed it?
2. Explain, line by line, what your total function does.
3. Which technique from the CSS techniques session did you use? Say what each property in it does on your page.

What is checked	Marks
No console errors during normal use	2
Tidy indentation and clear names	2
notes.txt, 2 marks per question	6

## Rules

You may use your class notes, the workshop slides and MDN. You may ask an AI tool to explain an error message, but not to write your code. Your instructor may ask you to explain any part of your project. Marks for a part you cannot explain can be taken away.

## Before you submit

- ☐ The page opens with no errors in the console.
- ☐ Every nav link lands on its section.
- ☐ You can add 10 tickets but not 11, and no count goes below 0.
- ☐ With the starting prices, 2 Regular and 3 Student shows ₦10,500 before the discount and ₦9,450 after it.
- ☐ Book with nothing chosen, with no name and with a short phone number, and you get the right message each time.
- ☐ Clear puts everything back to the start.
- ☐ notes.txt answers all three questions.

Zip the event folder, name the zip with your full name, for example ada-okafor-exam.zip, and submit it before the deadline.